

BTS 2 – 2024/2025

Projet 2 : Création d'un Serveur Web en Java

Préambule

Ce projet est à réaliser seul ou en binôme, **une documentation détaillée** est demandée, celle ci devra :

- Présenter votre projet
- Répondre aux **questions** ouvertes dans ce document
- Expliquer comment fonctionne votre projet de façon détaillée, un schéma d'enchaînement des méthodes appelées peut être utile.
- Pour chaque version réalisée (0.1, 0.2, etc.) vous mettrez en annexe de votre documentation l'intégralité du code correspondant à cette version.
- Donner toutes les références externes utilisées (liens vers les documentations, liens vers le protocole, liste des codes http implémentés etc.).

Consignes

Vos noms de propriétés doivent être en anglais et explicites. Les noms de vos méthodes doivent être en anglais et basées sur un verbe et décrire de façon explicite ce que fait la méthode.

Vous utiliserez la notation standard suivante :

Classes : camelCase avec première lettre en majuscule.

Propriétés : camelCase avec première lettre en minuscule

Méthodes : camelCase avec première lettre en minuscule

Faites attention à la comparaison de chaînes de caractères en Java.

*L'ensemble de votre code devra être en mesure de **gérer les exceptions** pour cela vous utiliserez la commande try !*

Etape 1 : Comprendre comment fonctionne un serveur Web

Introduction

Comme nous l'avons vu en cours un serveur Web est une application s'exécutant sur une machine connectée à Internet qui sert à consulter des pages html en utilisant le protocole http, cette application ouvre un port TCP/IP (par défaut le port 80 en http et 443 en https) et attend de recevoir des requêtes http, lorsqu'il reçoit une requête le serveur renvoie un message (par exemple 200, 404, 503, ...) et soit le contenu demandé, soit un message d'erreur correspondant au problème qu'il a rencontré.

Ce protocole de communication est défini dans le document RFC 2616 qui se trouve sur le site de l'IETF, organisme qui définit et gère l'ensemble des protocoles ouvert sur lesquels est construit Internet : ce document de référence est disponible à l'adresse :

<https://www.ietf.org/rfc/rfc2616.txt>

Même s'il est conseillé de lire ce document il est tout à fait possible de réaliser un serveur web fonctionnel sans l'avoir lu.

Comprendre les échanges http

Avant toute chose il est important de comprendre le fonctionnement du protocole http, notamment à quoi ressemble une requête envoyée par un navigateur et la forme de ce qui sera renvoyé par le serveur lorsqu'il reçoit une requête http.

La requête http

Pour analyser une requête http envoyée par un navigateur nous allons tout d'abord utiliser un outil nous permettant de lire et de voir ce qui est demandé par un navigateur (Chrome, Firefox, Edge, Safari, Opera, etc.)

L'outil que nous allons utiliser est fourni en standard dans tous les systèmes basés sur unix (Linux, BSD, Mac OS X, etc.), pour Windows nous allons installer un équivalent qui permettra de faire la même chose.

L'outil sur les systèmes basés sur Unix s'appelle netcat, on le lance (en ligne de commande) avec la commande nc

Pour les utilisateurs sous windows on installera ncat, un outil équivalent disponible sur <https://nmap.org/ncat/> une fois installé on le lancera (en ligne de commande) avec la commande ncat

Dans les 2 cas la syntaxe est la même.

Une fois l'outil installé nous allons pouvoir regarder à quoi ressemble une requête http telle qu'envoyée par votre navigateur.

Pour cela lancez la commande suivante dans un terminal ou une fenêtre DOS :

```
nc 0.0.0.0 -v -l 8080
```

ou

```
ncat 0.0.0.0 -v -l 8080
```

La commande dit à l'ordinateur de nous afficher dans le détail toutes les requêtes TCP/IP qu'il recevra sur le port 8080.

Note : on utilisera le port 8080 car le port 80 de votre ordinateur est déjà occupé par le logiciel Apache que vous utilisez pour faire du PHP.

Une fois la commande lancée passez sous votre navigateur préféré et rendez-vous à l'adresse :

<http://127.0.0.1:8080>

(on demande au navigateur de se rendre à l'adresse 127.0.0.1 et de se connecter sur le port 8080).

Retournez sous votre terminal ou fenêtre DOS, vous devez voir s'y afficher la requête qui a été envoyée par votre navigateur.

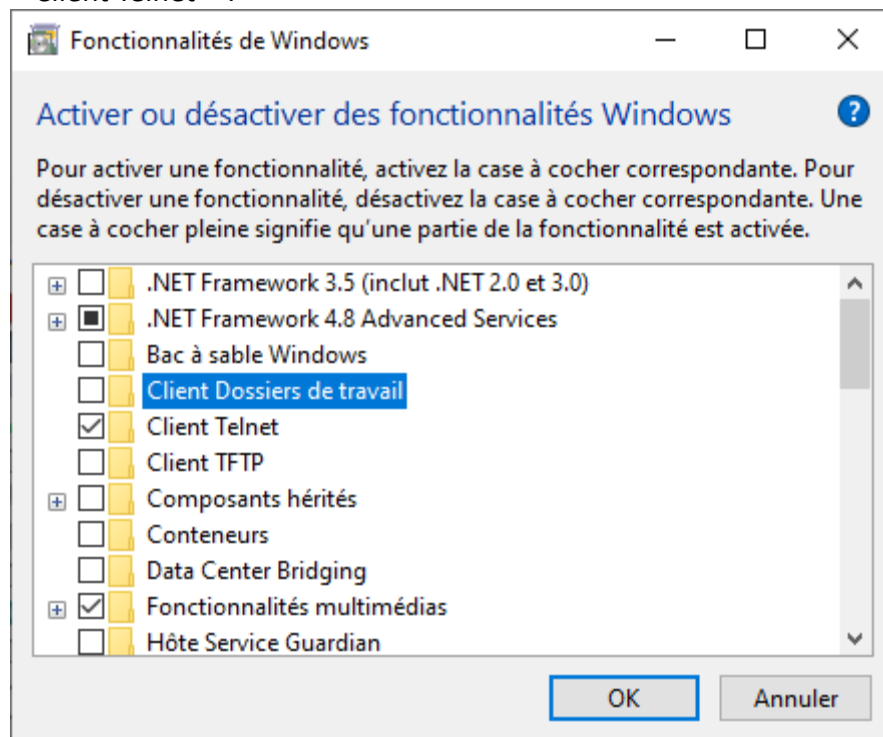
Question : Donnez le contenu de la requête envoyée par votre navigateur, si vous essayez d'analyser cette requête que contient-elle selon vous ? Vous devez constater que cette requête se compose 3 blocs différents, quels sont-ils ?

La réponse du serveur

Maintenant que vous savez à quoi ressemble une requête http et de quoi elle se compose il va falloir y répondre, d'une façon similaire à ce que nous avons fait pour comprendre la requête nous allons interroger un serveur web et lire sa réponse en mode console !

Pour cela nous utiliserons la commande telnet, cette commande permet d'interroger un serveur sur un port spécifié.

Elle est installée par défaut sur les systèmes Mac OS et linux, vous pouvez l'installer sur windows depuis le panneau de contrôle Fonctionnalités de Windows en cochant la case « Client Telnet » :



Elle s'utilise de la façon suivante pour interroger un serveur qui écoute sur un port http :

```
telnet <adresse IP du serveur> <port>
```

Par exemple tapez dans votre terminal ou ligne de commande DOS pour accéder au serveur web situé à l'adresse www.google.com

```
telnet www.google.com 80
```

vous devez avoir une réponse du type de celle ci :

```
Trying 2a00:1450:4007:80c::2004...
Connected to www.google.com.
Escape character is '^['.
```

Tapez ensuite :

```
GET /
```

Suivi de la touche entrée.

Cette commande demande au serveur auquel vous êtes connecté de vous envoyer le contenu de la page située à la racine du serveur web (/).

Vous devriez voir s'afficher une réponse ainsi que le contenu de la page html renvoyée par le serveur web situé à l'adresse <http://www.google.com>

Question : Là aussi si vous analysez la réponse reçue vous devez être capable de distinguer plusieurs blocs différents, essayez de les identifier et de comprendre ce qu'ils représentent.

Question 2 : Réalisez plusieurs tests, par exemple en appelant une page différente de la racine et qui ne se trouve pas sur le serveur, que constatez vous ?

Vous avez maintenant vu comment fonctionnait un échange entre un client et un serveur http et les différents types de réponse du serveur selon s'il trouvait la page demandée ou pas !

Il est important de bien comprendre cet échange et d'identifier les éléments qui ne servent qu'à la communication entre les 2 parties (client et serveur) qu'on appellera les entêtes http.

Fort de ces informations vous allez maintenant pouvoir commencer à coder votre serveur web !

Etape 2 : Implémenter un serveur web basique

Présentation du fonctionnement attendu

Vous allez implémenter une classe que nous appellerons `WebServerApplication`, celle ci contiendra votre méthode `main()` ainsi que toutes les propriétés et méthodes globales du projet.

La méthode `main()` appellera une classe `WebServer`, cette classe créera une instance de `ServerSocket` qui écoutera sur le port passé en paramètre et attendra les connexions des clients (voir la documentation de Java).

La classe `WebServer` contiendra 2 méthodes, une méthode `readRequest()` qui prendra comme argument un objet de type `Socket` et une méthode `sendResponse` qui prendra comme argument un objet de type `Socket`.

Pour chaque connexion reçue sur le port 8080 par votre application vous appellerez la méthode `readRequest()` qui affichera dans la console la requête reçue par l'application puis la méthode `sendResponse()` qui enverra au client une réponse disant que la requête a été lue et comprise par le serveur.

La méthode `main()` de `WebServerApplication` créera une instance de `WebServer` sur le port 8080.

Réalisation et test (version 0.1)

En vous appuyant sur la documentation de Java sur les Sockets réalisez ce qui a été présenté dans le chapitre précédent. Cela ne devrait pas vous poser de problèmes particuliers ! Une fois ceci réalisé vous pourrez tester votre application. Pour cela il suffit de lancer votre application `WebServerApplication` et de vous rendre depuis votre navigateur sur la page <https://127.0.0.1:8080>

Si tout fonctionne correctement votre navigateur doit afficher une page blanche sans symbole de chargement pendant que votre application affichera dans la console la requête reçue et la réponse envoyée !

Votre serveur web est maintenant capable de recevoir et de comprendre une requête http et de dire au client qui l'interroge qu'il a bien reçu et compris la requête. Il faut maintenant qu'il soit capable de renvoyer le contenu d'une page html !

Renvoyer une page html – version 0.2

Une fois que tout cela fonctionne vous allez modifier votre code pour que la méthode `sendResponse()` envoie au navigateur une page html, le navigateur devra être en mesure d'afficher cette page sans erreurs !

Pour cela créez une page html simple du type de celle ci :

```
< !DOCTYPE html>
<html>
  <head>
    <title>Page de test</title>
  </head>
  <body>
    <h1>Page de test de mon serveur web</h1>
    <p>Ceci est la page de test de mon serveur web, si vous pouvez lire ceci
c'est que tout fonctionne correctement</p>
  </body>
</html>
```

Enregistrez cette page dans un répertoire accessible par votre programme.

Modifiez votre code pour que la méthode `sendResponse()` renvoie son contenu lorsque le serveur web reçoit une requête.

Vous afficherez dans la console la requête reçue ainsi que le code html renvoyé par la méthode `sendResponse()`

Pour tester lancer l'application `WebServerApplication` et dirigez votre navigateur vers <http://127.0.0.1:8080> vous devez voir votre page html s'afficher sans erreur !

La version 0.2 de votre serveur web fonctionne, certes elle ne renvoie pour l'instant qu'une seule page html mais l'étape suivante (version 1.0) consistera à faire évoluer votre code pour lui permettre de renvoyer plusieurs pages différentes en fonction de la demande envoyée par le client, vous devrez prendre en compte les codes de réponses http les plus courant (200, 404, 503, etc.) et il faudra aussi rendre notre application multi tâches afin de pouvoir traiter plusieurs requêtes, un document vous donnant la marche à suivre vous sera communiqué ultérieurement !